Parcial práctico de Ingeniería de Software: La última transmisión

Integrantes: Valery Georgina Aurela Perez y Kevin Hernando Acuña Luna · **Curso:** Ingeniería de Software, Universidad Tecnológica de Bolívar (UTB)

Repositorio: https://github.com/KevinUtb/Parcial_2_Ing

Versión de este informe: anterior a la implementación de la solicitud de cambio (la sección 7 queda por completar). Anexo A: diagramas (Fig. 1 a 6). Anexo B: matriz de trazabilidad. Anexo C: captura del tablero. Los textos resaltados están pendientes.

1. Problema, actores y alcance

Un colectivo de escritores prepara una ficción interactiva para un festival universitario. Necesita cargar relatos desde archivos, ofrecer opciones condicionadas por lo ocurrido y saber qué corregir cuando un archivo tiene errores. Construimos un motor de consola en Python que interpreta las historias desde los datos, sin lógica propia de ninguna historia.

Actores. Lector (juega la historia), Escritor (redacta y edita el JSON sin programar) y Coordinadora del festival (necesita diagnósticos claros de los errores). **Alcance.** Incluye carga y validación de historias en JSON, escena actual con sus opciones habilitadas, elección con efectos, finales, bloqueo y reinicio. Excluye cuentas, base de datos, guardado entre ejecuciones, red, editor visual, audio, imágenes, lenguaje de expresiones y retroceso en la versión inicial (conflicto abierto, ver P01).

Riesgos y mitigación.

Riesgo	Mitigación	Evidencia
R1. Un archivo con errores provoca un cierre inesperado o una partida injugable.	Validar todo al cargar y rechazar con `ErrorContenido`; la consola informa cada error y no abre sesión.	`src/cargador.py`; pruebas U4 y CA03; RNF02. Riesgo residual: sección 6.
R2. La solicitud de cambio de la hora 6 obliga a tocar el motor y desordena requisitos, modelos y pruebas.	Motor sin datos de ninguna historia (RNF03), modelos en Mermaid fáciles de editar, pruebas automatizadas, tag de la versión inicial y tareas de cambio en el backlog.	`contenido/el_faro.json`; `tests/test_segunda_historia.py`; `docs/modelos/`

Prioridad de atributos de calidad. (1) Modificabilidad: la escritora debe cambiar escenas y decisiones sin tocar el programa, y es la necesidad más repetida del cliente (RNF03). (2) Claridad de errores: la coordinadora necesita saber qué corregir (RNF02). (3) Confiabilidad: el lector no debe ver cierres inesperados (RNF01). La facilidad de uso se atiende con una consola de números, pero no se mide; el diseño gráfico queda fuera del alcance.

Decisión de responsabilidad profesional. Usamos solo la biblioteca estándar de Python, sin dependencias de terceros, para no asumir obligaciones de licencia ni riesgos de cadena de suministro. La historia y sus reglas son datos del cliente y se usan sin alterar rutas ni reglas. Declaramos toda herramienta de apoyo, incluida la IA, y su verificación (sección 8).

2. Proceso y agilidad

Proceso. Elegimos un proceso incremental e iterativo con tablero Kanban, frente a un proceso en cascada: la ventana es de 12 horas con un cambio anunciado a la hora 6, y la cascada exigiría congelar requisitos y diseño antes de implementar. Las actividades: especificación en `docs/requisitos.md`; desarrollo en `src/`; validación con pruebas automatizadas y simulación de partidas; evolución con la versión inicial etiquetada, el análisis de impacto del cambio y la actualización de la matriz.

Backlog y tablero. El tablero tiene tres columnas (Pendiente, En proceso, Finalizado) y dos integrantes asignados. En la captura del Anexo C hay 7 tarjetas pendientes, 4 en proceso y 17 finalizadas. Entre las finalizadas: verificar el entorno, crear el repositorio, redactar RF01 a RF11, definir RNF01 a RNF03, registrar preguntas, supuestos y el defecto de E3, y los diagramas, componentes y decisiones. Al recibir la solicitud se ajustó el backlog con tarjetas de cambio: análisis de impacto y requisitos RF12 a RF14, implementar deshacer en motor y consola, y pruebas U6 con extensión de U5 e I1.

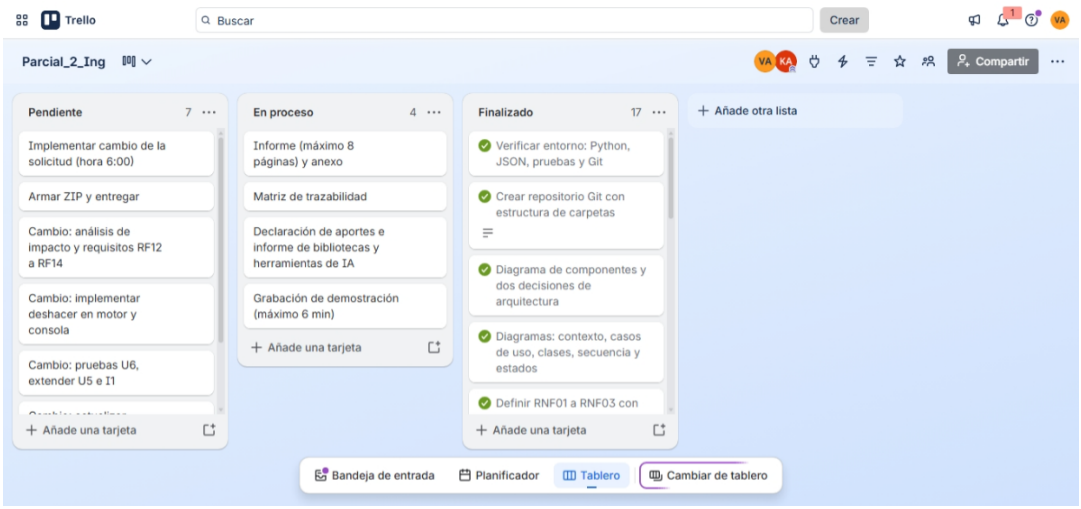


Figura A. Tablero Kanban del proyecto en estado intermedio (copia del Anexo C).

Prácticas aplicadas. (1) Tablero Kanban visible para ambos integrantes, con pocas tareas en proceso a la vez. (2) Pruebas automatizadas como criterio de avance, ejecutadas con `python -m unittest discover -s tests`. **Alternativa descartada:** Scrum con sprints y reuniones diarias, porque dos personas en 12 horas no justifican esas ceremonias.

Definición de terminado. Una tarea está terminada cuando (a) el cambio está en el repositorio con un mensaje de commit claro, (b) las pruebas pasan, (c) los documentos afectados (requisitos, modelos, README, matriz) están actualizados y (d) existe evidencia localizable.

Aprendizaje y acción de mejora. Un JSON guardado con un editor de Windows incluye BOM y fallaba al cargar. Acción: abrir los archivos con `utf-8-sig` y agregar `test_archivo_con_bom_se_carga`. Acción pendiente: validar los tipos de `id` y `destino` (sección 6).

3. Requerimientos

Los RF cubren las necesidades N01 a N06; cada uno tiene comportamiento observable y criterio de aceptación (texto completo en `docs/requisitos.md`).

ID	Requisito	Criterio de aceptación	Nec.
RF01	Cargar una historia desde un archivo JSON.	Con un archivo válido, la carga devuelve la historia con escena inicial y variables.	N01
RF02	Tomar escena inicial y variables iniciales del archivo.	Con la historia de referencia, la partida inicia en E1 con ambas variables en falso.	N01
RF03	Mostrar el texto de la escena actual.	Al iniciar o avanzar se muestra el texto de la escena activa.	N02
RF04	Mostrar solo las opciones habilitadas.	Con `conoce_origen` en falso, E3 no muestra «Activar con código».	N02
RF05	Al elegir una opción habilitada, aplicar efectos y avanzar al destino.	Desde E2, «Investigar archivo» lleva a E3 con `conoce_origen` en verdadero.	N03
RF06	Una opción inexistente o no habilitada no modifica el estado.	Enviar un ID no habilitado deja escena y variables iguales.	N03
RF07	Conservar las variables durante la partida.	Tras un efecto, la siguiente elección evalúa el valor actualizado.	N04
RF08	Identificar los finales y terminar la partida.	Al llegar a E4, E5 o E6, la sesión termina sin opciones.	N05
RF09	Reiniciar restaurando escena y variables iniciales.	Tras un final y un reinicio, vuelve a E1 con variables en falso.	N05
RF10	Rechazar un archivo inválido indicando causa y campo o ID.	Una copia con destino E99 falla con un mensaje que nombra E99.	N06
RF11	Informar cuando una escena no final no tiene opciones habilitadas.	Se muestra el bloqueo y la opción de reiniciar.	N06

Requisitos no funcionales medibles.

ID	Medida y meta	Procedimiento	Resultado y evidencia
RNF01 Confiabilidad	Excepciones no controladas: 0 en 1 000 partidas.	`python tests/simular_partidas.py`: 500 partidas aleatorias por historia (referencia y segunda), `random.seed(42)`.	CUMPLE: 1 000 partidas, 0 excepciones, 0 bloqueos, 0 sin terminar. `evidencia/rnf01.txt`
RNF02 Claridad de errores	100 % de las 9 categorías de error con mensaje de causa y campo o ID.	`tests/test_cargador.py`: un caso por categoría; cada prueba comprueba causa e identificador.	CUMPLE: 7 pruebas OK (10 variantes parametrizadas más archivo ilegible). `evidencia/rnf02.txt`

ID	Medida y meta	Procedimiento	Resultado y evidencia
RNF03 Modificabilidad	0 líneas de `src/` modificadas para cargar la segunda historia.	`git diff --stat` entre el commit anterior y el posterior a agregar `contenido/el_faro.json`, limitado a `src/`; y una prueba que verifica que `motor.py` no contiene IDs ni variables de las historias.	Prueba automática OK. PENDIENTE: salida vacía de `git diff --stat 3891d3f HEAD -- src/` en `evidencia/rnf03.txt`

Ejemplo de mensaje (RNF02): `Escena 'E3', opción 'cortar': destino inexistente 'E99'.`

Preguntas, supuestos y defectos.

- **P01 (pregunta abierta).** ¿Debe permitirse retroceder siempre? La escritora quiere retroceso permanente y el diseñador decisiones irreversibles. Acuerdo provisional: no implementar retroceso en la versión inicial, sin presuponer una decisión posterior del cliente.
- **P02 (supuesto).** Si una escena no final queda sin opciones habilitadas, se informa el bloqueo y se permite reiniciar (RF11).
- **P03 (pregunta).** Las variables no declaradas en condiciones o efectos se rechazan antes de iniciar la partida (RF10).
- **D01 (defecto y corrección).** La regla «E3 muestra exactamente dos opciones» dependía del filtrado y no estaba garantizada por la escena. Se agregó `test_e3_siempre_dos_opciones`, que comprueba ambos valores de `conoce_origen`.

4. Modelado

Los seis modelos están en el Anexo A, uno por archivo `.mmd` en `docs/modelos/`, con el vocabulario del código y de los requisitos (Historia, Escena, Opcion, Sesion, condiciones, efectos).

Fig.	Modelo	Qué muestra	RF
1	Contexto	Sistema, actores y fronteras: archivo JSON y consola.	RF01, RF10
2	Casos de uso	Cargar, validar, recibir errores, ver escena, elegir y reiniciar.	RF01 a RF11
3	Clases del dominio	`Sesion` y `ErrorContenido` son clases; Historia, Escena y Opcion son diccionarios leídos del JSON; `cargador` es un módulo.	RF01 a RF11
4	Secuencia: elegir una opción	De `consola.jugar` a `Sesion.elegir`, con el caso de opción no habilitada.	RF05, RF06, RF07
5	Estados de una sesión	Jugando, Final y Bloqueada, derivados de `es_final` y `bloqueada`; transiciones por elegir y reiniciar.	RF08, RF09, RF11
6	Componentes y capas	Interfaz, reglas y contenido, con las pruebas (sección 5).	todos

Coherencia: los nombres de métodos y atributos de las figuras 3 a 5 coinciden con `src/motor.py` y `src/cargador.py`. Los modelos corresponden a la versión inicial y se actualizan tras el cambio (sección 7).

5. Arquitectura

La Fig. 6 muestra tres capas. **Contenido:** `contenido/*.json`, que define escenas, opciones, condiciones y efectos. **Reglas:** `cargador.py` lee y valida el JSON (`cargar_historia`, `validar_historia`, `ErrorContenido`) y `motor.py` mantiene el estado de la partida (`Sesion`). **Interfaz:** `consola.py` lee el teclado, imprime y reporta errores de contenido. `consola.py` importa cargador y motor; `motor.py` no importa nada; las pruebas usan cargador, motor y el JSON real.

AD-01. Contenido en archivos JSON, fuera del código.

- *Contexto:* la escritora quiere cambiar escenas sin pedir cambios al programa (N01), y el proyecto solo usa la biblioteca estándar.
- *Alternativas:* historias como estructuras de Python en el código; YAML (requiere una biblioteca externa); un lenguaje de expresiones propio (fuera de alcance).
- *Decisión:* JSON con el contrato de la guía, leído con `json`.
- *Consecuencias:* agregar o cambiar una historia no toca `src/`. Costos: JSON no admite comentarios y es sensible a comas y comillas; los editores de Windows pueden agregar BOM, por lo que se lee con `utf-8-sig`.

AD-02. Validar todo el contenido al cargar y devolver los errores juntos.

- *Contexto:* la coordinadora necesita saber qué corregir y el lector no debe encontrar un cierre inesperado (N06).
- *Alternativas:* fallar en el primer error; validar durante la partida; que `Sesion` valide su propia historia.
- *Decisión:* `validar_historia` devuelve una lista de mensajes y `cargar_historia` lanza `ErrorContenido` con el atributo `errores`; la consola los imprime y termina con código 1 sin crear una sesión.
- *Consecuencias:* una historia que llega a `Sesion` ya fue validada, y la validación se prueba sin archivos. Costos: «todos a la vez» tiene límites (sección 6) y el motor depende de esa validación previa.

Separación demostrada con la segunda historia. `contenido/el_faro.json` (cuatro escenas, la variable `tiene_linterna`, condiciones y un efecto) se carga con el mismo motor mediante `python src/consola.py contenido/el_faro.json`. `tests/test_segunda_historia.py` comprueba su carga, el cambio de opciones por la condición, sus dos finales y que `motor.py` no contiene IDs ni variables de ninguna historia.

6. Implementación y pruebas

Tecnología. Python 3 (probado con 3.12.3), interfaz de consola. Bibliotecas: `json` y `unittest`; en pruebas y scripts también `os`, `sys`, `random`, `collections`, `tempfile` y `copy`, todas de la biblioteca estándar de Python (licencia PSF). No hay dependencias de terceros ni nada que instalar.

Decisión de diseño detallado: responsabilidades y acoplamiento. `Sesion` recibe un diccionario ya validado y no importa el cargador ni la consola. Su método `elegir(opcion_id)` solo acepta las opciones devueltas por `opciones_habilitadas()`, de modo que la regla de habilitación vive en un único lugar: aunque se envíe directamente el identificador de una opción no habilitada, devuelve `False` y deja el estado intacto (CA02). `es_final` y `bloqueada` son propiedades derivadas, no estado guardado, así que no pueden desincronizarse. La consola solo traduce el número escrito a un ID. Costo: `Sesion` confía en que la historia fue validada.

Pruebas automatizadas. Última ejecución: 18 pruebas, todas OK (`evidencia/ultima_ejecucion.txt`).

Caso	Prueba	Qué comprueba
U1	`test_motor.test_u1_condiciones`	Las condiciones cumplidas y no cumplidas habilitan solo las opciones correspondientes.
U2	`test_motor.test_u2_eleccion_valida`	Una elección válida aplica efectos y avanza al destino correcto.
U3	`test_motor.test_u3_opcion_invalida_no_cambia_estado`	Opción inexistente, de otra escena o no habilitada (CA02) deja el estado intacto.
U4	`test_cargador.test_u4_*` y `test_ca03_*`	Las 9 categorías de error de contenido, con datos parametrizados, y CA03.
U5	`test_motor.test_u5_reiniciar`	Reiniciar restaura escena y variables iniciales.
I1	`test_integracion.test_i1_tres_finales_desde_archivo_real`	Archivo real y motor juntos, desde la carga hasta cada uno de los tres finales.
Otras	`test_bloqueo_en_escena_no_final`, `test_e3_siempre_dos_opciones`, `test_archivo_con_bom_se_carga`, `test_segunda_historia` (4)	RF11, D01, defecto del BOM y la segunda historia.

Verificación del producto. Los tres finales se recorren por consola con las entradas `1,1,1` (E4), `1,2,1` (E5) y `1,2,2` (E6), y se verifican automáticamente en I1. Se cargó la segunda historia con el mismo motor y se mostró el error del destino `E99` (CA03).

Fallos y limitaciones pendientes.

- Un `destino` o un `id` de opción que sea una lista (no texto) lanza `TypeError` en lugar de `ErrorContenido`.
- Si falta un campo de primer nivel o tiene un tipo incorrecto, solo se informan esos errores y no los de las escenas.
- No se verifica la alcanzabilidad de los finales; el bloqueo solo se informa durante la partida.
- RF03 (mostrar el texto) y la lista numerada de la consola se verifican manualmente, sin prueba automática.

7. Gestión del cambio y trazabilidad

Solicitud de cambio (hora 6). Según las tarjetas del tablero, el cambio consiste en deshacer en el motor y la consola, con los requisitos RF12 a RF14, la prueba U6 y la extensión de U5 e I1. PENDIENTE: completar con el texto de la solicitud y el resultado.

Elemento	Registro
Versión de partida	PENDIENTE: tag y hash del commit de la versión inicial
Contenido del cambio	PENDIENTE: resumen de la solicitud
Requisitos, modelos, componentes y pruebas afectados	PENDIENTE Previsibles: P01, RF06, RF09, `Sesion`, `consola.jugar`, figuras 3 a 5, U5 e I1.
Ajuste del backlog	Tarjetas de cambio agregadas al tablero (Anexo C).
Implementación y nuevo escenario unitario	PENDIENTE: archivos y prueba U6
Evidencia de funcionamiento	PENDIENTE

Trazabilidad. El Anexo B relaciona cada RF (RF01 a RF11, más los del cambio) y los tres RNF con las figuras del Anexo A, los archivos de `src/` y `contenido/` que los atienden y las pruebas o archivos de evidencia con su ruta. RF03 solo tiene evidencia manual.

8. Aportes, fuentes y herramientas de apoyo

Integrante	Aportes
Valery Georgina Aurela Perez	Verificación del entorno (con Kevin); diagramas Mermaid (contexto, casos de uso, clases, secuencia, estados y componentes) y decisiones de arquitectura; informe; matriz de trazabilidad; commits de esos entregables (3891d3f); parte del armado del ZIP. Los diagramas, el informe y la matriz se elaboraron con apoyo de la herramienta de IA declarada abajo.
Kevin Hernando Acuña Luna	Verificación del entorno (con Valery); repositorio y estructura de carpetas; requisitos RF01 a RF11, RNF01 a RNF03, preguntas, supuestos y defecto de E3; cargador y validador (62be7b5); motor `Sesion` y consola (c6c9fce); pruebas U1 a U5, bloqueo, validaciones, CA03, BOM e I1 (e7cdb00, 2c7c819, a93ac9c); segunda historia El faro apagado y simulador de partidas del RNF01; parte del armado del ZIP.

Fuentes y herramientas. Biblioteca estándar de Python (`json`, `unittest`); Git y GitHub para versionado; Mermaid para los diagramas; tablero Kanban en Trello para el backlog. El texto de la historia y sus reglas provienen de la guía del parcial.

Herramientas de IA. Se usó el asistente Claude (Anthropic) para redactar borradores de los diagramas Mermaid, el README, este informe y la matriz de trazabilidad a partir del código del repositorio y de `docs/requisitos.md`. Los resultados se contrastaron con el código y con la ejecución de las pruebas y del simulador de partidas (RNF01). El código fuente, las pruebas y la segunda historia son trabajo propio de la pareja.